

PracticalWork — презентация результатов

Library & Reports microservices · .NET 10 · Clean Architecture · Unit tests + Coverage

Покрытие (строки): Domain 69.23%, Application 61.5%, Infrastructure 0.7%

Артефакты: REPORT.md, COVERAGE.md, docs/DEPLOYMENT.md, docs/DOCUMENTATION.md

1. Цели и задачи

- Привести решения Library и Reports к Clean Architecture и SOLID
- Устранить антипаттерны: бизнес-логика в контроллерах, прямой DbContext, жёсткие зависимости
- Добавить unit-тесты и измерение покрытия (HTML + XML отчёт)
- Сформировать документацию: XML docs, Swagger, README, развёртывание

2. Исходная архитектура (проблемы)

API / Controllers

бизнес-логика в action
прямые вызовы MinIO/DbContext
валидация неполная

Infrastructure

EF/Redis/RabbitMQ без портов
создание подключений внутри классов
сложно подменять в тестах

Domain

часть ответственности смешана
workflow нарушал SRP

Риски: высокая связность, низкая тестируемость, сложные изменения.

3. Целевая архитектура (Clean Architecture)

API

тонкие контроллеры
DTO + валидация
вызов use-cases

Application

use-cases / workflows
порты (интерфейсы)
оркестрация сценариев

Infrastructure

EF/Redis/MinIO/RabbitMQ
реализация портов
DI в composition root

Плюсы: тестируемость, заменяемость инфраструктуры, локализация изменений.

4. Примеры рефакторинга (было/стало)

- Контроллеры: логика вынесена в сервисы (IActivityLogService, IBorrowService)
- Reports consumer: IActivityLogIngestionService + ack/nack вместо autoAck
- WeeklyReportWorkflow разделён на компоненты (SRP)
- DI: AddLibraryApplication / AddReportsApplication

5. Структура unit-тестов

- tests/PracticalWork.Library.Tests: Services, Validation, Models
- tests/PracticalWork.Reports.Tests: Services, Validation
- Всего тестов: 67
- Подход: моки портов (репозитории/кэш/email/minio), проверка use-cases

6. Покрытие тестами (Coverlet/TestPlatform)

Слой	Покрытие строк	Строки
Domain	69.23%	126 / 182
Application	61.5%	671 / 1091
Infrastructure	0.7%	2 / 284

Отчёты: [coverage-report/index.html](#) (HTML), [coverage-report/xml/*.cobertura.xml](#) (XML)

Запуск: `scripts/Generate-CoverageReport.cmd` (обходит ExecutionPolicy).

7. Миграция на .NET 10

- Проекты переведены на net10.0
- Сборка: `dotnet build PracticalWork.Library.sln`
- Тесты: `dotnet test PracticalWork.Library.sln`

8. Созданная документация

- XML docs: GenerateDocumentationFile=true (Directory.Build.props)
- Swagger: /swagger для обоих сервисов
- README.md: быстрый старт, API, структура решения
- docs/DEPLOYMENT.md: Docker, порты, миграции, troubleshooting
- docs/DOCUMENTATION.md: сводка документации
- COVERAGE.md: запуск покрытия и отчёты

9. Выводы

- Clean Architecture: API тонкий, use-cases в Application, инфраструктура за портами
- Unit-тесты покрывают ключевые сценарии, есть HTML отчёт покрытия
- Документация и развёртывание подготовлены для двух сервисов
- Следующий шаг: integration-тесты (PostgreSQL/Redis/RabbitMQ) для роста Infrastructure coverage